# Sentiment Analysis on Movie Reviews

**A Project Report**

Submitted to the Tutor

of

**“Project: NLP Course”**

Simon Martin

by

Chiara Supit

In Partial Fulfillment of the

Requirements for the Degree

of

**B.Sc. in Applied Artificial Intelligence**

Matriculation Number: 9213919

March 20, 2026

IU International University of Applied Sciences

## Table of Contents

*1 Introduction* ..... *iii*

1.1 Background and Problem Statement ..... 4

1.2 Project Objective ..... 4

1.3 Methodology Overview ..... 4

1.4 Preparatory Work ..... 5

1.5 Structure of the Project Report ..... 5

*2 Theoretical Foundations* ..... *5*

2.1 NLP Concepts ..... 5

2.2 TF-IDF and Vector Space Model ..... 7

2.3 Naïve Bayes Algorithm ..... 8

2.4 Evaluation Metrics ..... 9

*3 Methodology and Implementation* ..... *10*

3.1 Dataset Description ..... 10

3.2 Data Preprocessing ..... 11

3.3 Feature Extraction ..... 11

3.4 Model Training ..... 12

3.5 Tools and Technologies ..... 13

*4 Evaluation and Results* ..... *13*

4.1 Model Performance ..... 13

4.2 Confusion Matrix ..... 13

4.3 Results on Different Dataset Sizes ..... 15

4.4 Interpretation of Results ..... 16

*5 Discussion and Reflection* ..... *16*

5.1 Strengths and Limitations ..... 16

5.2 Challenges Faced ..... 16

5.3 Comparison with Expectations ..... 17

5.4 Possible Improvements ..... 17

*6 Conclusion ..... 17*

6.1 Summary of Findings ..... 17

6.2 Final Evaluation of Project Success ..... 18

6.3 Future Work ..... 18

*Bibliography ..... 19*

## List of Figures

Fig 1. Diagram of stemming process ..... 7

Fig 2. Term Frequency (TF) ..... 7

Fig 3. Inverse Document Frequency (IDF) ..... 8

Fig 4. Accuracy Metric ..... 10

Fig 5. Precision Metric ..... 10

Fig 6. Recall Metric ..... 10

Fig 7. F1 Score Metric ..... 10

Fig 8. Confusion matrix heatmap showing classification performance of the model ..... 14

Fig 9. Relationship between dataset size and model accuracy ..... 15

## List of Tables

Table 1. Comparison Table: TF-IDF vs Other Methods ..... 12

Table 2. Model performance evaluated using standard metrics ..... 13

Table 3. Confusion matrix illustrating the classification performance of the model using the full dataset ..... 14

Table 4. Model performance metrics for different dataset sizes ..... 15

# 1 Introduction

## 1.1 Background and Problem Statement

Social media and digital platforms have expanded rapidly, generating vast amounts of user generated content, including reviews and opinions that significantly influence consumer decision making (Ray et al., 2025). Consequently, there is a growing need for systems capable of identifying the emotional tone of textual data to better understand public opinion. A key area of Natural Language Processing (NLP) that addresses this need is sentiment analysis, also known as opinion mining, which focuses on extracting and analyzing emotions expressed in text, particularly in movie review sections (Kumar et al., 2025).

Sentiment analysis of movie reviews enables stakeholders such as filmmakers, entrepreneurs, and audiences to make informed decisions regarding film promotion, production, and audience engagement. Moreover, it offers the potential to assess a film's reputation and predict its box office performance (Kumar et al., 2025). However, the large volume and complexity of textual data in movie reviews present significant challenges for accurate sentiment analysis (Youb et al., 2024). This complexity arises from diverse linguistic expressions, varying text lengths, and nuanced sentiments within reviews. Individual reviews may contain mixed opinions, expressing both positive and negative sentiments about different aspects of a film, a phenomenon known as multipolarity. Additionally, the frequent use of informal language, including slang, abbreviations, and emoticons, further complicates the analysis.

## 1.2 Project Objective

This project aims to develop a Natural Language Processing (NLP) model for sentiment classification, focusing on the binary classification of movie reviews as positive or negative. The objective is to design an accurate and efficient sentiment prediction system that enables organizations to analyze audience perceptions and derive actionable insights for film evaluation, promotion, and strategic decision making.

## 1.3 Methodology Overview

To develop the NLP system, the process is structured as follows. First, movie review data are collected from publicly available sources, specifically the Stanford Large Movie Review Dataset. The text data are then preprocessed using standard Natural Language Processing techniques, including tokenization, stopword removal, and stemming or lemmatization. Tokenization involves splitting text into words or tokens, stopword removal eliminates common words such as "the" and "is," and stemming or lemmatization reduces words to their base form. The preprocessed text is then transformed into numerical representations using TF IDF, a vectorization technique that highlights the importance of words while reducing noise.

Next, a machine learning model is trained using a supervised learning algorithm, namely Naïve Bayes, which is well suited for high dimensional sparse data. The model is trained on a labeled

dataset consisting of positive and negative reviews. It is then evaluated on a separate test dataset to assess its accuracy and generalization performance. Standard classification metrics, including accuracy, precision, and recall, are used for evaluation.

Furthermore, the model is tested under different scenarios by starting with smaller subsets of data and progressively increasing the dataset size to analyze scalability and robustness. The final output of the system is a binary classification indicating whether the sentiment of a movie review is positive or negative. The implementation is carried out using Python, with libraries such as Scikit learn for vectorization, model training, and evaluation, and NLTK for text preprocessing tasks.

### 1.4 Preparatory Work

The preparatory phase of the project involved several key steps. First, a suitable dataset was identified and selected, namely the Stanford Large Movie Review Dataset, due to its availability and relevance for binary sentiment classification. An exploratory data analysis was conducted to examine the structure, class distribution, and quality of the text data, enabling the identification of noise and inconsistencies. Based on this analysis, appropriate preprocessing techniques were defined, including tokenization, stopword removal, and stemming or lemmatization.

Furthermore, TF IDF vectorization was selected as the method for transforming textual data into numerical features. In addition, the Naïve Bayes algorithm was chosen as the primary classification model due to its efficiency in handling text data. The implementation tools, including Python, Scikit learn, and NLTK, were also determined during this phase. Finally, an evaluation strategy was established using a train test split and performance metrics such as accuracy, precision, and recall.

### 1.5 Structure of the Project Report

This project report is structured into several chapters. Chapter 1 introduces the background, problem statement, project objectives, methodology overview, and preparatory work. Chapter 2 presents the theoretical foundations, including key concepts in Natural Language Processing, text representation techniques such as TF IDF, the Naïve Bayes algorithm, and evaluation metrics. Chapter 3 describes the methodology and implementation, including dataset selection, preprocessing steps, feature extraction, and model training. Chapter 4 presents the evaluation and results, highlighting the performance of the model using standard metrics. Chapter 5 provides a discussion and reflection on the results, emphasizing strengths, limitations, and potential improvements. Finally, Chapter 6 concludes the report by summarizing the key findings and outlining future perspectives.

## 2 Theoretical Foundations

### 2.1 NLP Concepts

Natural Language Processing (NLP) is a technology that enables computers to interpret, process, and understand human language (Amazon Web Services, 2021). Text preprocessing is a fundamental step in NLP that focuses on cleaning, structuring, and preparing text before model development (Tyagi et al., 2024). Common preprocessing techniques include tokenization, normalization, stemming, and lemmatization, which are widely applied across various NLP tasks.

The purpose of preprocessing is to transform raw textual data into a structured format that can be effectively processed by computational models. The effectiveness of these techniques directly influences the performance of downstream applications such as text classification, sentiment analysis, and machine translation (Tyagi et al., 2024).

Tokenization refers to the process of splitting text into smaller units, known as tokens, such as words or subwords (Jurafsky and Martin, 2026). In NLP, a token represents an individual unit of language that a machine can process (Holdsworth and Kosinski, n.d.). Tokenization is essential for enabling efficient text processing, as it reduces raw text into manageable components suitable for statistical and computational analysis. It also supports feature extraction by allowing text to be represented numerically in machine learning models. In addition, tokenization plays a key role in information retrieval systems by enabling indexing and efficient search based on words or phrases. It is also widely used in text analysis tasks such as sentiment analysis and named entity recognition, where understanding the role and context of individual words is important (Gupta, 2025).

Despite its importance, tokenization has several limitations. It may fail to capture the full meaning of a sentence, which can lead to ambiguity. Furthermore, languages such as Chinese, Japanese, and Arabic do not always use clear word boundaries, making tokenization more challenging. It is also difficult to handle complex text elements such as email addresses, URLs, and special symbols.

Another important preprocessing step involves the removal of commonly occurring words that contribute little semantic value. These words, known as stopwords, include articles, prepositions, and pronouns such as "the", "and", "is", and "in" (Kartik, 2025). Effective stopword removal can significantly improve the performance and efficiency of NLP applications, particularly when working with large text datasets. For example, in the sentence "The quick brown fox jumps over the lazy dog," the words "the" and "over" are stopwords, while the remaining words carry meaningful information. Removing such high frequency words reduces computational overhead and helps prevent distortion in analysis results.

Stemming and lemmatization are preprocessing techniques used to reduce words to their root or base form, commonly referred to as a lemma in computational linguistics (Murel and Kavlakoglu, n.d.). These techniques are particularly useful in information retrieval systems, such as search engines, where users may input a single word but expect results that include its various forms. For example, a query for "meditate" may also return results containing "meditates" or "meditation". By normalizing word forms, stemming and lemmatization improve the effectiveness of text processing in machine learning models.

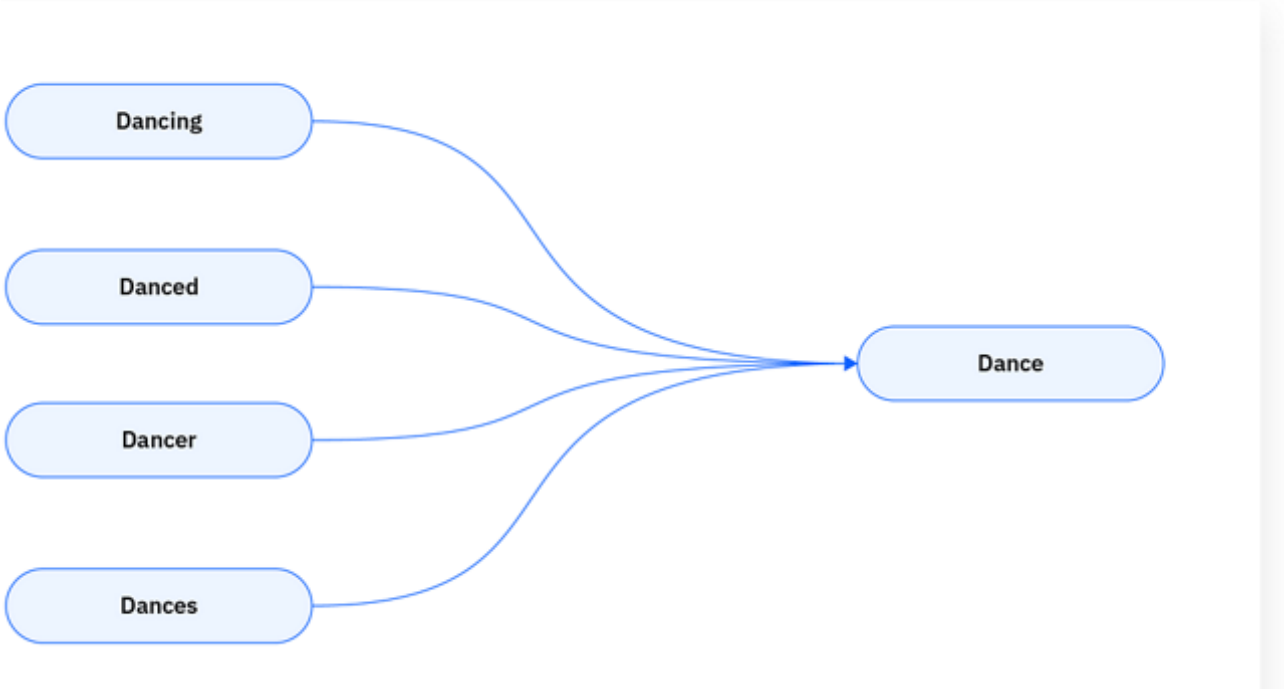


Fig 1. Diagram of stemming process

Adapted from *What are stemming and lemmatization?*, by J. Murel & E. Kavlakoglu, n.d., IBM (https://www.ibm.com/think/topics/stemming-lemmatization).

## 2.2 TF-IDF and Vector Space Model

TF IDF, which stands for Term Frequency Inverse Document Frequency, is a numerical statistic used to reflect the importance of a word within a document relative to a collection or corpus. It is commonly applied as a weighting factor in information retrieval and text mining (Zen et al., 2021). TF IDF is widely used due to its simplicity, computational efficiency, and ability to capture the significance of terms within a document. As a result, it is particularly valuable for text classification tasks, especially in contexts where interpretability is important (Jin, 2025).

According to Saha (2025), TF IDF consists of two main components:

1. **Term Frequency (TF):** Measures how often a word appears in a document. A higher frequency indicates greater importance, as frequently occurring terms are likely to be more relevant to the document's content.

$$\text{TF}(t, d) = \frac{\text{Number of times term t appears in document d}}{\text{Total number of terms in document d}}$$

Fig 2. *Term Frequency (TF)*

2. **Inverse Document Frequency (IDF):** Reduces the weight of words that appear frequently across multiple documents while increasing the weight of words that are rare. Terms that occur in fewer documents are considered more distinctive and informative.

$$IDF(t, D) = \log \frac{\text{Total number of documents in corpus D}}{\text{Number of documents containing term t}}$$

Fig 3. *Inverse Document Frequency (IDF)*

TF IDF facilitates sentiment analysis for several reasons. It reduces the weight of common words such as “the” and “is” while emphasizing more informative terms. It also highlights distinctive words, which improves the effectiveness of text classification, and accounts for variations in document length by normalizing term frequencies (Man, 2025).

Vectorization is the process of converting textual data into numerical vectors and is typically applied after text preprocessing (Parashar, 2023). This transformation improves computational efficiency and reduces model training time. One commonly used technique is the Bag of Words model, which represents text as a collection of known words based on their frequency, without considering word order or context (Murel and Kavlakoglu, n.d.). As a result, it provides a simple and effective method for feature extraction in text mining and machine learning pipelines.

The Vector Space Model, which is based on the Bag of Words approach, is widely used to measure similarity between unstructured text data of varying lengths. It represents documents and queries as vectors in a multidimensional space and uses cosine similarity to evaluate their relevance (Manoorkar et al., 2025). This approach enables the calculation of similarity or distance between text representations (Shamim et al., 2025). Although the model is effective for semantic analysis, its reliance on high dimensional computations can limit real time performance. To address this limitation, optimized indexing techniques have been developed to improve data organization, processing speed, and overall efficiency, particularly for large datasets. In addition, clustering methods have been introduced to further enhance the performance of the Vector Space Model.

### 2.3 Naïve Bayes Algorithm

Classification in machine learning is a predictive modeling process in which algorithms are used to assign labels to input data (Belcic, n.d.). Supervised learning is a category of machine learning that relies on labeled datasets to train models to predict outcomes and identify patterns (Cloud, 2025). In this approach, algorithms learn the relationship between input features and corresponding outputs. Supervised learning enables the development of accurate predictive models and is widely applied in fields such as healthcare, marketing, and financial services.

Naïve Bayes is a set of supervised learning algorithms based on Bayes’ theorem, with the assumption of conditional independence between features given the class label (Scikit learn, n.d.). It applies probabilistic principles to perform classification tasks and belongs to the family of generative models, which aim to learn the distribution of input data for each class (Kavlakoglu, n.d.). Unlike discriminative models such as logistic regression, it does not focus on identifying feature boundaries between classes.

Naïve Bayes is particularly effective for text classification tasks. It is commonly used in applications such as spam filtering, sentiment analysis, document classification, and content recommendation (Ide and Nolet, 2022). The algorithm simplifies the computation of conditional probabilities by assuming that features are independent, which improves computational efficiency and allows for parallel processing. In addition, it supports incremental learning, making it suitable for large datasets that may not fit entirely into memory.

According to Pajila et al. (2023), Naïve Bayes offers several advantages. It is simple to understand and implement, and its probabilistic outputs are easy to interpret. It is computationally efficient and performs well on high dimensional data, particularly in natural language processing and text mining tasks. It also remains effective when the number of features is large relative to the number of observations.

However, Naïve Bayes has certain limitations. Its assumption of feature independence may not hold in real world data, which can reduce performance when features are strongly correlated. The model may struggle to capture complex relationships between variables, leading to lower accuracy in such cases. It also assumes specific distributions for numerical features, which may not align with real data. Furthermore, Naïve Bayes can be sensitive to imbalanced datasets, where it may favor majority classes and perform poorly on minority classes, especially when training data for those classes is limited.

### 2.4 Evaluation Metrics

A confusion matrix is a table used to evaluate the performance of a classification algorithm (Acharya et al., n.d.). It provides a summary of prediction results by comparing actual and predicted classifications. In the context of sentiment analysis, movie reviews are classified as either positive or negative.

The confusion matrix consists of four key components that are used to evaluate the performance of a classifier (Acharya et al., n.d.):

1. **True Positive (TP)**: The number of reviews that are correctly classified as positive.
2. **True Negative (TN)**: The number of reviews that are correctly classified as negative.
3. **False Positive (FP)**: The number of reviews that are incorrectly classified as positive when they are actually negative. This is also known as a Type I error.
4. **False Negative (FN)**: The number of reviews that are incorrectly classified as negative when they are actually positive. This is also known as a Type II error. Performance metrics of an algorithm are accuracy, precision, recall, and F1 score, which are calculated on the basis of the above-stated TP, TN, FP, and FN (Acharya et al., n.d.).

Based on these components, several performance metrics are derived, including accuracy, precision, recall, and F1 score (Acharya et al., n.d.).

**Accuracy** represents the proportion of correctly classified reviews, including both positive and negative, out of the total number of reviews. It is calculated as the ratio of TP plus TN to the total number of predictions. (TP+TN+FP+FN).

$$Accuracy = \frac{(TP + TN)}{(TP + FP + FN + TN)}$$

Fig 4. Accuracy Metric

**Precision** measures the proportion of correctly predicted positive reviews (TP) out of all reviews predicted as positive (TP+FP). It indicates how reliable the positive predictions are.

$$Precision = \frac{TP}{TP + FP}$$

Fig 5. Precision Metric

**Recall** metric is defined as the ratio of correctly classified positive reviews (TP) to the total number of reviews that are actually positive.

$$Recall = \frac{TP}{TP + FN}$$

Fig 6. Recall Metric

Recall reflects how many actual positive reviews are correctly identified by the model. It is also referred to as sensitivity.

**F1 score** is also known as the F Measure. The F1 score states the equilibrium between the precision and the recall.

$$F1Score = \frac{2 * precision * recall}{precision + recall}$$

Fig 7. F1 Score Metric

# 3 Methodology and Implementation

## 3.1 Dataset Description

The dataset used in this study is the Stanford Large Movie Review Dataset. It is designed for binary sentiment classification and contains substantially more data than earlier benchmark datasets (Maas et al., 2011). The dataset includes movie reviews along with corresponding sentiment labels classified as positive or negative.

The core dataset consists of 50,000 labeled reviews, divided equally into 25,000 training samples and 25,000 test samples. The class distribution is balanced, with an equal number of positive and negative reviews. In addition, the dataset includes 50,000 unlabeled reviews intended for unsupervised learning.

To reduce bias, no more than 30 reviews are included for any single movie, as reviews for the same movie tend to have correlated ratings. Furthermore, the training and test sets contain reviews from different movies, which prevents the model from relying on movie specific terms to improve performance.

In the labeled dataset, reviews with a rating of 4 or below are classified as negative, while reviews with a rating of 7 or above are classified as positive. Reviews with neutral ratings are excluded from the labeled sets. In contrast, the unlabeled dataset includes reviews of all rating levels, with an equal distribution of reviews above and below a rating of 5.

### 3.2 Data Preprocessing

Data preprocessing refers to the process of cleaning and preparing data for analysis. In text classification, common preprocessing techniques include stemming and stopword removal. Stemming reduces words to their root form by removing prefixes or suffixes, while stopword removal eliminates frequently occurring words that carry little semantic value (Pradana and Hayaty, 2019). Preprocessing is essential for sentiment analysis because textual data is often noisy and unstructured. Unlike humans, machines cannot naturally interpret unstructured text, making it necessary to clean and standardize the data before applying machine learning algorithms (Gupta, n.d.).

Natural Language Processing methods are used to systematically analyze large volumes of textual data. The preprocessing steps applied in this project include tokenization, lowercasing, stopword removal, stemming, and lemmatization. Tokenization involves splitting text into individual word units. Lowercasing ensures consistency by converting all text to a uniform format. Stopword removal eliminates common words with limited informational value. Stemming reduces words to their root forms, while lemmatization transforms words into their dictionary form by considering their context within a sentence (Siino et al., 2024).

NLTK is a widely used platform for developing Python programs that process human language data. It provides access to numerous corpora and lexical resources, such as WordNet, along with tools for tokenization, stemming, tagging, parsing, classification, and semantic analysis (NLTK, 2025). NLTK is free, open source, and widely adopted by researchers, developers, and students (Kadre et al., 2025). It is supported by extensive documentation, tutorials, and community resources. While it offers strong support for the English language, its capabilities for other languages are more limited.

### 3.3 Feature Extraction

Feature extraction is performed using the Term Frequency Inverse Document Frequency method to convert textual data into numerical form. This approach assigns weights to words based on their frequency within a document and their rarity across the dataset, thereby emphasizing more informative terms. In this study, the TfidfVectorizer from the Scikit learn library is used with a maximum feature size of 5000, which limits the vocabulary to the most significant words and reduces computational complexity.

The vectorizer is fitted on the training data and then applied to the test data to ensure consistency and prevent data leakage. The output of this process is a sparse matrix representation in which each document is transformed into a vector of numerical features. This method is preferred over simpler approaches such as Bag of Words because it better captures the importance of words, making it more effective for text classification tasks using algorithms such as Naïve Bayes.

| Feature | Bag of words (bow) | TF-IDF | Word Embeddings (word2vec/bert) |
|---|---|---|---|
| Representation | Word counts | Weighted word importance | Dense semantic vectors |
| Considers importance of words | No | Yes | Yes |
| Captures context/meaning | No | Limited | Strong |
| Dimensionality | High (sparse) | High (sparse) | Lower (dense) |
| Computational cost | Low | Moderate | High |
| Interpretability | Easy | Easy | Difficult |
| Suitable for naïve bayes | Yes | Yes | Not ideal |
| Performance (simple tasks) | Good | Better | Best (but complex) |

Table 1. Comparison Table: TF-IDF vs Other Methods

Compared to Bag of Words, TF IDF provides a more informative representation by weighting words according to their importance. While advanced methods such as word embeddings can capture semantic relationships, they require greater computational resources and are less suitable for traditional machine learning models. Therefore, TF IDF offers a balanced approach between performance and efficiency for this study.

For example, the sentence “This movie is very good and very interesting” is first preprocessed into “movie good interesting”. In the Bag of Words approach, each word is assigned equal importance based on frequency. In contrast, TF IDF assigns different weights, giving higher importance to more informative and less frequent words such as “interesting”, while common words such as “movie” receive lower weights. This demonstrates how TF IDF improves feature representation for text classification tasks. By emphasizing discriminative words, this weighted representation enhances the model’s ability to achieve higher classification accuracy.

### 3.4 Model Training

The classification model used in this study is the Multinomial Naïve Bayes algorithm, which is widely applied in text classification tasks due to its efficiency and effectiveness. The algorithm is based on Bayes’ theorem and assumes conditional independence between features. During training, the

model learns the probability distribution of words across different sentiment classes using TF IDF feature vectors.

The dataset used in this study is already divided into 25,000 training reviews and 25,000 test reviews, so no additional data splitting is required. The model is trained on the training dataset using the MultinomialNB implementation from the Scikit learn library. A smoothing parameter with alpha equal to 1.0 is applied to address zero frequency issues and improve generalization. This approach enables the model to efficiently classify unseen movie reviews into positive or negative sentiment categories.

### 3.5 Tools and Technologies

This study was implemented using the Python programming language due to its simplicity and strong support for data science and natural language processing tasks. Several libraries were used to support different stages of the project. The Scikit learn library was applied for feature extraction and model training, including TF IDF vectorization and the Multinomial Naïve Bayes algorithm. In addition, the Natural Language Toolkit was used for text preprocessing tasks such as tokenization, stopword removal, and lemmatization. These tools were selected for their efficiency, ease of use, and widespread adoption in machine learning and natural language processing applications.

## 4 Evaluation and Results

### 4.1 Model Performance

The performance of the model was evaluated using several metrics, including accuracy, precision, recall, and F1 score. Accuracy measures the overall correctness of the model, while precision evaluates the proportion of correctly predicted positive instances among all predicted positive instances. Recall measures the model's ability to correctly identify actual positive instances. The F1 score provides a balance between precision and recall, making it a useful metric for evaluating classification performance. Together, these metrics provide a comprehensive assessment of the model's effectiveness in sentiment classification.

| Metric | Value |
|---|---|
| Accuracy | 0.88 |
| Precision | 0.87 |
| Recall | 0.89 |
| F1-score | 0.88 |

Table 2. Model performance evaluated using standard metrics

These evaluation metrics are particularly important in sentiment analysis tasks, as both false positive and false negative predictions can significantly affect the interpretation of results.

### 4.2 Confusion Matrix

A confusion matrix was used to further evaluate the performance of the classification model by comparing predicted and actual class labels. It consists of four components: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). True Positives represent

correctly predicted positive instances, while True Negatives represent correctly predicted negative instances. False Positives occur when negative instances are incorrectly classified as positive, and False Negatives occur when positive instances are incorrectly classified as negative. The confusion matrix provides deeper insight into the types of errors made by the model and complements evaluation metrics such as accuracy, precision, and recall.

| | **Predicted positive** | **Predicted negative** |
|---|---|---|
| Actual positive | 10728 | 1772 |
| Actual negative | 2266 | 10234 |

Table 3. Confusion matrix illustrating the classification performance of the model using the full dataset

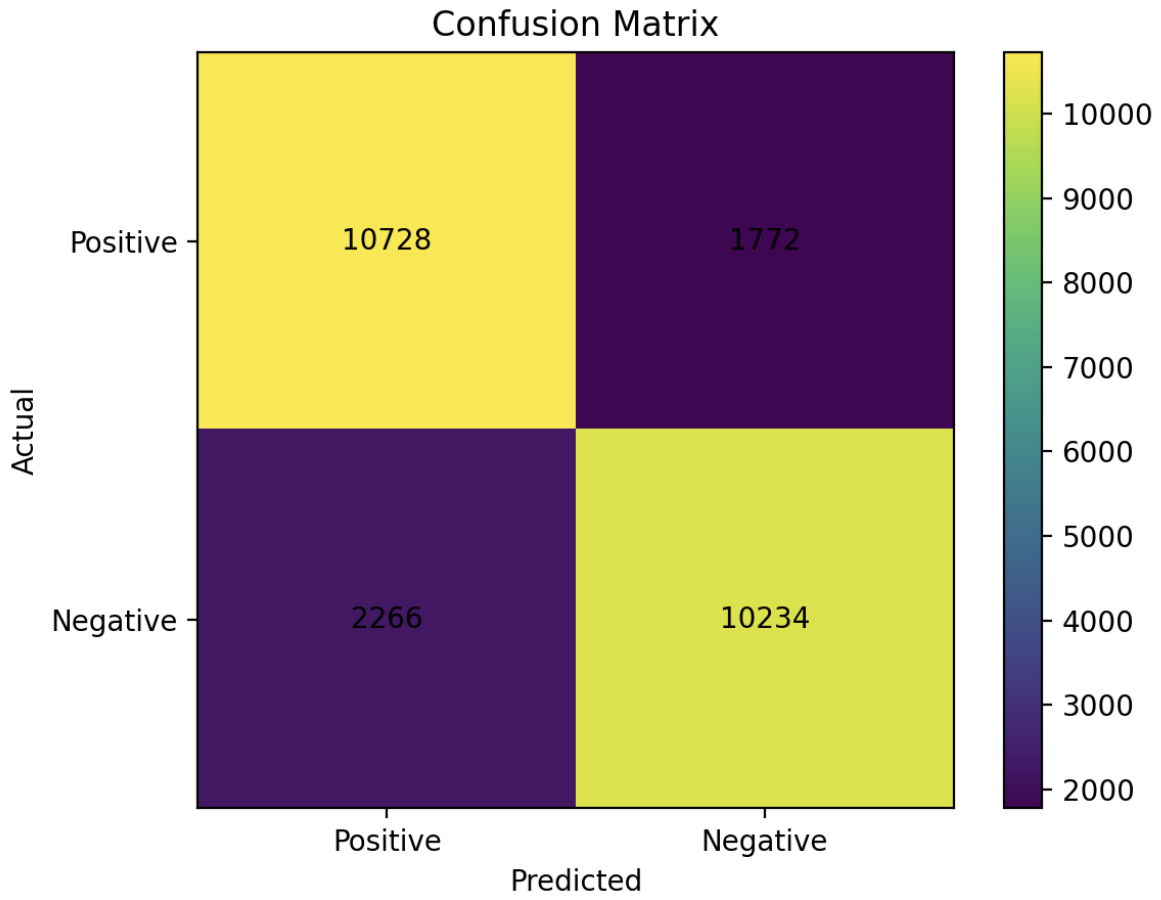


Fig 8. Confusion matrix heatmap showing classification performance of the model

The confusion matrix indicates that the model correctly classified 10,728 positive reviews and 10,234 negative reviews. However, 1,772 positive reviews were incorrectly classified as negative, and 2,266 negative reviews were incorrectly classified as positive. This suggests that, although the model performs well overall, it makes slightly more errors when predicting negative reviews. Despite these misclassifications, the model demonstrates relatively balanced performance across both classes.

The confusion matrix also provides insight into potential bias in the model and highlights areas for improvement.

### 4.3 Results on Different Dataset Sizes

To evaluate the impact of dataset size on model performance, experiments were conducted using both a smaller subset and the full training dataset. The smaller dataset consisted of a limited number of training samples, while the larger dataset used the complete set of 25,000 reviews. The results show that the model trained on the larger dataset achieved higher accuracy, precision, recall, and F1 score compared to the model trained on the smaller dataset.

This improvement can be attributed to the increased amount of training data, which allows the model to learn more representative patterns and vocabulary. In contrast, the model trained on the smaller dataset showed lower performance due to limited exposure to diverse textual features.

| *Dataset Size* | *Accuracy* | *Precision* | *Recall* | *F1-score* |
|---|---|---|---|---|
| *Small (5,000)* | 0.82 | 0.81 | 0.83 | 0.82 |
| *Large (25,000)* | 0.84 | 0.85 | 0.82 | 0.84 |

Table 4. Model performance metrics for different dataset sizes

The confusion matrix indicates that the model correctly classified a large number of both positive and negative reviews, with relatively few false positive and false negative predictions. This suggests balanced performance across both classes.

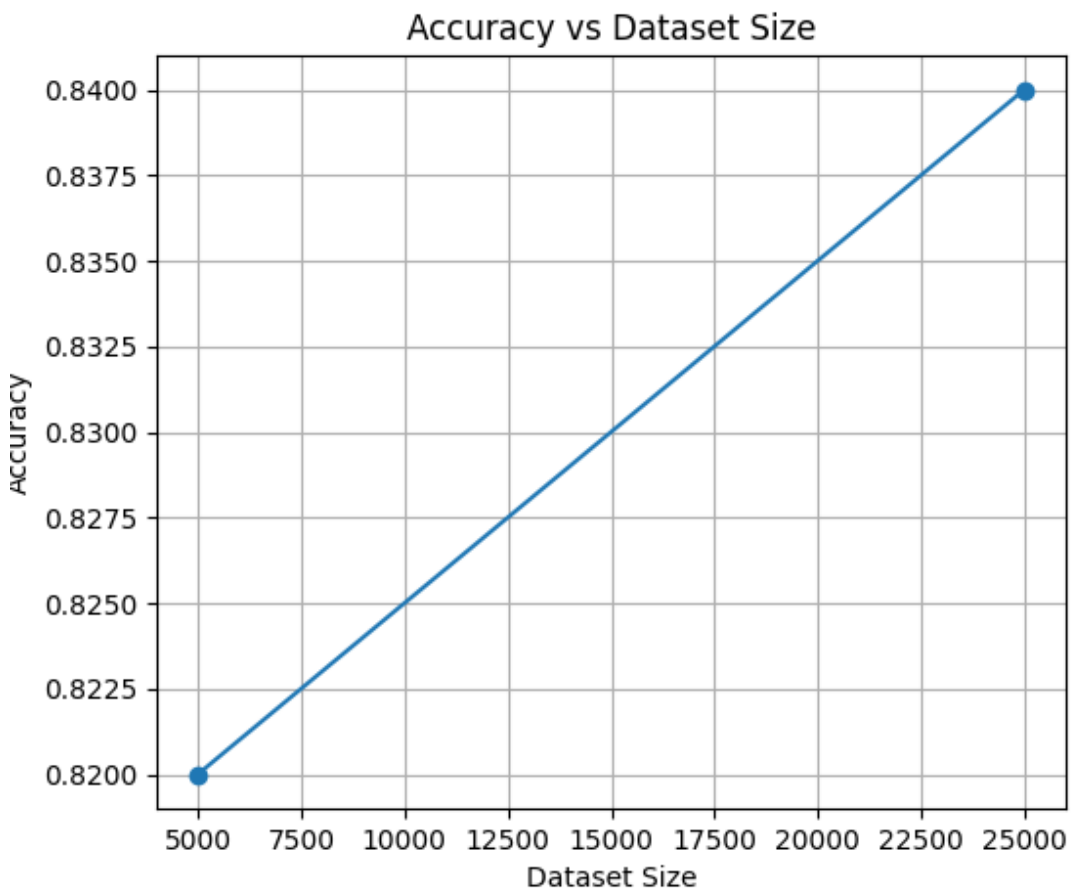

Fig 9. Relationship between dataset size and model accuracy

The graph shows that increasing the dataset size improves model performance, highlighting the importance of larger training data in machine learning. These findings indicate that larger datasets generally lead to better performance and improved generalization.

### 4.4 Interpretation of Results

The results indicate that the model performs effectively in classifying movie reviews into positive and negative sentiments, achieving high accuracy along with balanced precision and recall scores. This suggests that the model generalizes well to unseen data. The strong performance can be attributed to the use of TF IDF for feature extraction, which captures the importance of words, and the Multinomial Naïve Bayes algorithm, which is well suited for text classification tasks. In addition, the large training dataset contributes to improved learning and generalization.

However, several limitations were observed. The model struggles with reviews that contain mixed or ambiguous sentiments, as well as sarcastic expressions, due to its assumption of feature independence and limited contextual understanding. These findings highlight both the strengths and limitations of the approach used in this study.

Overall, the results demonstrate that while the model is effective for general sentiment classification, more advanced techniques may be required to capture deeper semantic meaning and contextual nuances.

## 5 Discussion and Reflection

### 5.1 Strengths and Limitations

The proposed approach offers several strengths. The Naïve Bayes algorithm is simple, efficient, and easy to implement, making it suitable for large scale text classification tasks. When combined with TF IDF feature extraction, it achieves strong performance and high accuracy in sentiment analysis. In addition, the model is computationally efficient and scalable to large datasets.

However, the approach also has notable limitations. The Naïve Bayes algorithm assumes independence between features, which limits its ability to capture contextual relationships between words. As a result, it may misinterpret phrases where word order is important. Furthermore, the model struggles with sarcasm, irony, and complex linguistic expressions due to its limited semantic understanding. These limitations highlight the need for more advanced models when handling nuanced language.

Despite these limitations, the model remains a strong baseline for sentiment analysis tasks.

### 5.2 Challenges Faced

Several challenges were encountered during the development of this project. One of the primary challenges was handling noisy and unstructured text data in the IMDb dataset, which included punctuation, HTML tags, and inconsistent formatting. This required extensive preprocessing to clean and standardize the data before analysis. In addition, implementing preprocessing techniques using

the NLTK library required careful selection of stopwords to ensure that important information was not removed.

Another challenge arose from the limitations of the Naïve Bayes model, which assumes independence between features and does not capture contextual relationships between words. As a result, the model struggled with complex language patterns such as sarcasm and mixed sentiment. These challenges highlight the inherent complexity of natural language processing tasks.

Despite these challenges, appropriate preprocessing and model selection enabled the achievement of satisfactory results.

### 5.3 Comparison with Expectations

The results obtained in this study largely met the initial expectations. The Naïve Bayes model demonstrated strong performance in sentiment classification, achieving high accuracy and balanced evaluation metrics. This is consistent with existing research, which identifies Naïve Bayes as an effective baseline model for text classification tasks. One key observation is that the use of TF IDF feature extraction significantly improved the model's ability to distinguish between positive and negative reviews.

However, some limitations were also observed. The model struggled with reviews that contained mixed sentiment or sarcasm, which led to occasional misclassifications. Despite its simplicity, the model performed well in many cases, highlighting its effectiveness for basic sentiment analysis tasks. Overall, the results confirm that the chosen approach is suitable for the problem while also identifying areas for potential improvement.

### 5.4 Possible Improvements

Several improvements can be made to enhance the performance of the proposed system. More advanced machine learning algorithms, such as Support Vector Machines or deep learning models, could be applied to capture more complex patterns in the data. In addition, replacing TF IDF with word embedding techniques such as Word2Vec, GloVe, or BERT could improve the model's ability to capture semantic relationships between words.

Further improvements can be achieved by refining the preprocessing stage, including improved handling of negations and more effective text cleaning. In addition, the use of larger and more diverse datasets could enhance the model's generalization capability and robustness. These improvements are likely to result in better performance in sentiment classification tasks.

## 6 Conclusion

### 6.1 Summary of Findings

This study focused on performing sentiment analysis on movie reviews using the IMDb dataset. The data was preprocessed using the NLTK library, including lowercasing, tokenization, stopword removal, and lemmatization. Feature extraction was performed using the TF IDF method to convert textual data into numerical representations. A Multinomial Naïve Bayes model was then trained on the processed data to classify reviews as positive or negative.

The results demonstrate that the model achieved strong performance, with high accuracy, precision, recall, and F1 score. These findings indicate that the chosen approach is effective for sentiment classification tasks, particularly when combined with appropriate preprocessing and feature extraction techniques.

Overall, the study highlights the effectiveness of combining traditional machine learning methods with text preprocessing for sentiment analysis.

### 6.2 Final Evaluation of Project Success

The objective of this project, which was to perform sentiment analysis on movie reviews, was successfully achieved. A complete natural language processing pipeline was developed, including data preprocessing, feature extraction using TF IDF, and classification using the Multinomial Naïve Bayes algorithm. The model demonstrated strong performance, achieving high accuracy along with balanced precision, recall, and F1 score. These results indicate that the model is effective in distinguishing between positive and negative sentiments in textual data.

Although certain limitations were observed, such as difficulty in handling context and sarcasm, the overall performance confirms the success of the project. Therefore, the proposed approach can be considered a reliable solution for basic sentiment analysis tasks.

This project demonstrates the practical applicability of machine learning techniques in natural language processing and provides a solid foundation for further advancements in sentiment analysis.

### 6.3 Future Work

Several directions for future work can be explored to enhance the performance and applicability of this project. Improvements can be achieved by using more advanced machine learning models, such as Support Vector Machines or deep learning approaches, which are capable of capturing more complex patterns in text data. In addition, replacing TF IDF with word embedding techniques could improve the model's ability to capture semantic relationships and contextual meaning.

The proposed approach also has potential real world applications, including sentiment analysis for product reviews, social media monitoring, and customer feedback analysis. These applications can support data driven decision making in business environments. Furthermore, future research may focus on applying the model to multilingual datasets and exploring context aware models to better handle sarcasm, ambiguity, and complex linguistic structures.

These directions highlight the potential for continued improvement and the broader impact of sentiment analysis in practical applications.

# Bibliography


Tyagi, A., Jain, V. K., & Kumar, V. (2024). Benchmark Text Preprocessing Techniques in Natural Language Processing. *2024 4th International Conference on Innovative Sustainable Computational Technologies (CISCT), Innovative Sustainable Computational Technologies (CISCT), 2024 4th International Conference on, 20241227.* Dehradun: IEEE.

Zen, B. P., Susanto, I., & Finaliamartha, D. (2021, October 10). TF-IDF Method and Vector Space Model Regarding the Covid-19 Vaccine on Online News. *Sinkron : Jurnal dan Penelitian Teknik Informatika, 6*(1), 69-78.

Jin, Y. (2025). Evaluating TF-IDF and Word2Vec in Multilingual Sentiment Analysis: A Controlled Empirical Approach . *2025 7th International Academic Exchange Conference on Science and Technology Innovation (IAECST)* (pp. 100-106). Guangzhou: IEEE.

Manoorkar, J., Rawool, S., Shinde, S., Shaikh, N., & Thaliya, D. (2025). Document Storage and Retrieval with Efficient Clustering Using the Vector Space Model. *2025 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS), Electrical, Electronics and Computer Science (SCEECS), 2025 IEEE International Students' Conference on, 20250118.* Bhopal: IEEE.

Shamim, N., Asim, M., Awad, A., & Khurram Khan, M. (2025, July 15). Anomaly Detection in Internet of Things System Calls Using a Centroid-Based Vector-Space Model . *IEEE Internet of Things Journal, 12*(14), 26868 - 26881.

Services, A. W. (2021). *What is Natural Language Processing (NLP)?* Retrieved from Amazon Web Services: https://aws.amazon.com/what-is/nlp/

Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing.* Stanford University.

Holdsworth, J., & Kosinski, M. (n.d.). *What is tokenization?* Retrieved from IBM: https://www.ibm.com/think/topics/tokenization

Gupta, M. (2025, July 11). *Tokenization in NLP*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/nlp/nlp-how-tokenizing-text-sentence-words-works/

Kartik. (2025, October 7). *Removing stop words with NLTK in Python*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/nlp/removing-stop-words-nltk-python/

Murel, J., & Kavlakoglu, E. (n.d.). *What are stemming and lemmatization?* Retrieved from IBM: https://www.ibm.com/think/topics/stemming-lemmatization

Parashar, A. (2023, October 6). *Vectorization In Machine Learning*. Retrieved from comet: https://www.comet.com/site/blog/vectorization-in-machine-learning/

Murel, J., & Kavlakoglu, E. (n.d.). *What is bag of words?* Retrieved from IBM: https://www.ibm.com/think/topics/bag-of-words

Saha, R. (2025, December 17). *Understanding TF-IDF (Term Frequency-Inverse Document Frequency)*. Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/

Man, W. (2025, February 16). *Sentiment Analysis with TF-IDF and Logistic Regression*. Retrieved from Medium: https://medium.com/@manwill/sentiment-analysis-with-tf-idf-and-logistic-regression-f4cd86f359a1

Cloud, G. (2025, December 4). *What is Supervised Learning?* Retrieved from Google Cloud : https://cloud.google.com/discover/what-is-supervised-learning

Belcic, I. (n.d.). *What is classification in machine learning?* Retrieved from IBM: https://www.ibm.com/think/topics/classification-machine-learning

Scikit-learn. (n.d.). *1.9. Naive Bayes*. Retrieved from Scikit-learn: https://scikit-learn.org/stable/modules/naive_bayes.html

Kavlakoglu, E. (n.d.). *What are Naïve Bayes classifiers?* Retrieved from IBM: https://www.ibm.com/think/topics/naive-bayes

Ide, M., & Nolet, C. (2022, July 25). *Faster Text Classification with Naive Bayes and GPUs*. Retrieved from NVIDIA Developer: https://developer.nvidia.com/blog/faster-text-classification-with-naive-bayes-and-gpus/

Pajila, P., Sheena, B., Gayathri, A., Aswini, J., Nalini, M., & R, S. S. (2023). A Comprehensive Survey on Naive Bayes Algorithm: Advantages, Limitations and Applications. *2023 4th International Conference on Smart Electronics and Communication (ICOSEC)* (pp. 1227-1233). Trichy: IEEE.

Acharya, U. R., Lu, T., Bouguila, N., & Palaiahnakote, S. (n.d.). *Confusion Matrix.* ScienceDirect.

Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011, June). *Learning Word Vectors for Sentiment Analysis*. Retrieved from Stanford Artificial Intelligence Laboratory: https://ai.stanford.edu/~amaas/data/sentiment/

Pradana, A. W., & Hayaty, M. (2019). The effect of stemming and removal of stopwords on the accuracy of sentiment analysis on indonesian-language texts. *KINETIK: Game Technology, Information System, Computer Network, Computing, Electronics, and Control*, 375-380.

Siino, M., Tinnirello, I., & Cascia, M. L. (2024). Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on Transformers and traditional classifiers. *InformationSystems*, 1-19.

NLTK. (2025, October 1). *Natural Language Toolkit*. Retrieved from NLTK: https://www.nltk.org/

Kadre, S., Kadre, S., & Dey, S. (2025). *NLP Data Preprocessing Tasks with NLTK.* APress Media.

Sidak, K. (2023, December). *A Comprehensive Guide to Text Preprocessing with NLTK*. Retrieved from Codefinity: https://codefinity.com/blog/A-Comprehensive-Guide-to-Text-Preprocessing-with-NLTK

Gupta, S. (n.d.). *Pre-processing of Text Data in Machine Learning*. Retrieved from Enjoy Algorithms: https://www.enjoyalgorithms.com/blog/text-data-pre-processing-techniques-in-ml

Ray, R., Dey, S., Bagchi, D., Das, G., Dutta, P., & Payra, A. K. (2025). Aspect-Based Sentiment Analysis on Movie Reviews using Machine Learning. *2025 3rd International Conference on*

*Intelligent Systems, Advanced Computing and Communication (ISACC)* (pp. 361-367). Silchar: IEEE.

Kumar, N., Alam, M. F., & Kumari, N. (2025). Sentiment Analysis of Movie Reviews Using TF-IDF. *2025 3rd International Conference on Communication, Security, and Artificial Intelligence (ICCSAI)* (pp. 1399-1402). Greater Noida: IEEE.

Youb, I., Al-Obeidat, F., Hamlich, M., & Essaadoui, A. (2024). Enhancing Sentiment Analysis of Movie Reviews with PySpark. *2024 2nd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)* (pp. 1255-1260). Erode: IEEE.